

Core Language Working Group "ready" Issues for the October, 2021 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4885](#).

1249. Cv-qualification of nested lambda capture

Section: 7.5.5 [expr.prim.lambda] **Status:** ready **Submitter:** James Widman **Date:** 2011-03-02

Consider the following example:

```
void f(int i) {
    auto l1 = [i] {
        auto l2 = [&i] {
            ++i;    // Well-formed?
        };
    };
}
```

Because the `l1` lambda is not marked as `mutable`, its `operator()` is `const`; however, it is not clear from the wording of 7.5.5 [expr.prim.lambda] paragraph 16 whether the captured member of the enclosing lambda is considered `const` or not.

Proposed resolution (August, 2021):

Change 7.5.5.3 [expr.prim.lambda.capture] paragraph 14 as follows:

If a *lambda-expression* `m2` captures an entity and that entity is captured by an immediately enclosing *lambda-expression* `m1`, then `m2`'s capture is transformed as follows:

- **if** `m1` captures the entity by copy, `m2` captures the corresponding non-static data member of `m1`'s closure type; **if `m1` is not mutable, the non-static data member is considered to be const-qualified.**
- **if** `m1` captures the entity by reference, `m2` captures the same entity captured by `m1`.

1724. Unclear rules for deduction failure

According to 13.10.3 [temp.deduct] paragraph 8,

If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written using the substituted arguments.

Presumably the phrase “if written” refers to rewriting the template declaration *in situ* with the substituted arguments, rather than writing that type or expression at some arbitrary location, e.g.,

```
void g(double) = delete;

template<class T> auto f(T t) -> decltype(g(t));

void g(int);

void h() {
    typedef int T;
    T t = 42;
    g(t);    // Ok (I “wrote the substituted arguments”, and it seems fine)
    f(42);   // Presumably substitution is meant to fail.
}
```

Perhaps a clearer formulation could be used?

Proposed resolution (August, 2021):

Change 13.10.3.1 [temp.deduct.general] paragraph 8 as follows:

If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written **in the same context** using the substituted arguments.

1726. Declarator operators and conversion function

Presumably the following example is intended to be ill-formed:

```
struct A {
    (*operator int*());
};
A a;
int *x = a; // Ok?
```

It is not clear, however, which rule is supposed to reject such a *member-declaration*.

Proposed resolution (September, 2021):

Change 11.4.8.3 [class.conv.fct] paragraph 1 as follows, splitting the paragraph into three paragraphs:

~~A member function of a class *x* with a name of the form~~

conversion-function-id:
operator *conversion-type-id*

conversion-type-id:
*type-specifier-seq conversion-declarator*_{opt}

conversion-declarator:
*ptr-operator conversion-declarator*_{opt}

~~shall have no parameters and~~ A declaration whose *declarator-id* is a *conversion-function-id* shall declare a function or function template, or an explicit instantiation or specialization of a function template, that has no parameters and is a non-static member of a class or class template **x**; it specifies a conversion from **x** to the type specified by the *conversion-type-id*, interpreted as a *type-id* (9.3.2 [dcl.name]). Such functions are called *conversion functions*.

~~A decl-specifier in the decl-specifier-seq of a conversion function (if any) shall be neither a defining-type-specifier nor static.~~ In a conversion function declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be *inline*, *constexpr*, *constexpr*, *virtual*, or an *explicit-specifier*. The type of the conversion function (9.3.4.6 [dcl.fct]) is “*noexcept*_{opt} function taking no parameter *cv-qualifier-seq*_{opt} *ref-qualifier*_{opt} returning *conversion-type-id*”.

A conversion function is never used to convert a (possibly cv-qualified) object to the (possibly cv-qualified) same object type (or a reference to it), to a (possibly cv-qualified) base class of that type (or a reference to it), or to cv void.¹⁰² [Example 1:...

1733. Return type and value for operator= with *ref-qualifier*

Section: 9.5.2 [dcl.fct.def.default] **Status:** ready **Submitter:** James Widman **Date:** 2013-08-09

9.5.2 [dcl.fct.def.default] paragraph 1 specifies that an explicitly-defaulted function shall

have the same declared function type (except for possibly differing *ref-qualifiers* and except that in the case of a copy constructor or copy assignment operator, the parameter type may be “reference to non-const τ ”, where τ is the name of the member function's class) as if it had been implicitly declared...

This allows an example like

```
struct A {  
    A& operator=(A const&) && = default;  
};
```

but forbids

```
struct B {  
    B&& operator=(B const&) && = default;  
};
```

which seems backward.

In addition, _N4750_15.8 [class.copy] paragraph 22 only specifies the return value for implicitly-declared copy/move assignment operators, not for explicitly-defaulted ones.

Proposed resolution (August, 2021):

1. Change 11.4.6 [class.copy.assign] paragraph 6 as follows:

The implicitly-declared copy/move assignment operator for class `x` has the return type `x&`; ~~it returns the object for which the assignment operator is invoked, that is, the object assigned to.~~
An implicitly-declared copy/move assignment operator is an inline public member of its class.

2. Add the following as a new paragraph following 11.4.6 [class.copy.assign] paragraph 13:

The implicitly-defined copy assignment operator for a union `x` copies the object representation (6.8 [basic.types]) of `x`. If the source and destination of the assignment are not the same object, then for each object nested within (6.7.2 [intro.object]) the object that is the source of the copy, a corresponding object `o` nested within the destination is created, and the lifetime of `o` begins before the copy is performed.

The implicitly-defined copy/move assignment operator for a class returns the object for which the assignment operator is invoked, that is, the object assigned to.

[Note: The first point in the issue, that of the relationship between the ref-qualifier and the return type, will be referred to EWG for consideration. The draft resolution above addresses only the second point of the issue.]

2484. char8_t and char16_t in integral promotions

Section: 7.3.7 [conv.prom] **Status:** ready **Submitter:** Richard Smith **Date:** 2021-04-01

According to 7.3.7 [conv.prom] paragraphs 1-2,

A prvalue of an integer type other than `bool`, `char16_t`, `char32_t`, or `wchar_t` whose integer conversion rank (6.8.5 [conv.rank]) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

A prvalue of type `char16_t`, `char32_t`, or `wchar_t` (6.8.2 [basic.fundamental]) can be converted to a prvalue of the first of the following types that can represent all the values of its underlying type: `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of its underlying type, a prvalue of type `char16_t`, `char32_t`, or `wchar_t` can be converted to a prvalue of its underlying type.

Because of its omission from the list of excluded types (perhaps as an oversight when it was added), `char8_t` is handled in the first paragraph. However, `char16_t` falls into the second paragraph, even though it is guaranteed to be convertible to `int` or `unsigned int`. This seems inconsistent, so perhaps `char8_t` should be moved to the second paragraph or `char16_t` moved to the first?

Notes from the August, 2021 teleconference:

`char8_t` should be handled by the second paragraph by including it in all three lists of types in the two paragraphs.

Proposed resolution (August, 2021):

Change 7.3.7 [conv.prom] paragraphs 1 and 2 as follows:

A prvalue of an integer type other than `bool`, `char8_t`, `char16_t`, `char32_t`, or `wchar_t` whose integer conversion rank (6.8.5 [conv.rank]) is less than the rank of `int` can be converted to a prvalue of type `int` if `int` can represent all the values of the source type; otherwise, the source prvalue can be converted to a prvalue of type `unsigned int`.

A prvalue of type `char8_t`, `char16_t`, `char32_t`, or `wchar_t` (6.8.2 [basic.fundamental]) can be converted to a prvalue of the first of the following types that can represent all the values of its underlying type: `int`, `unsigned int`, `long int`, `unsigned long int`, `long long int`, or `unsigned long long int`. If none of the types in that list can represent all the values of its underlying type, a prvalue of type `char8_t`, `char16_t`, `char32_t`, or `wchar_t` can be converted to a prvalue of its underlying type.

2486. Call to noexcept function via noexcept(false) pointer/lvalue

Section: 7.6.1.3 [expr.call] **Status:** ready **Submitter:** Jiang An **Date:** 2021-03-27

According to 7.6.1.3 [expr.call] paragraph 6,

Calling a function through an expression whose function type is different from the function type of the called function's definition results in undefined behavior.

This restriction should exempt calling a noexcept function where the function type of the expression is identical except that it is `noexcept(false)`.

In addition, 7.6.1.9 [expr.static.cast] paragraph 7 currently forbids `static_cast` from converting a function pointer or member function pointer from `noexcept(false)` to `noexcept`:

The inverse of any standard conversion sequence (7.3 [conv]) not containing an lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), function-to-pointer (7.3.4 [conv.func]), null pointer (7.3.12 [conv.ptr]), null member pointer (7.3.13 [conv.mem]), boolean (7.3.15 [conv.bool]), or function pointer (7.3.14 [conv.fctptr]) conversion, can be performed explicitly using `static_cast`.

This restriction should also be relaxed, allowing binding a `constexpr` reference to the result of the reversed conversion.

Notes from the August, 2021 teleconference:

CWG agreed that it should be permitted to call a noexcept function via an expression that is `noexcept(false)`; since the implicit conversion is allowed, the failure to allow the call is clearly just an oversight. The question of whether to allow the `static_cast` in the inverse direction, as well as whether to allow calling a `noexcept(false)` function via a `noexcept` expression (which would result in undefined behavior only if the function actually threw an exception) was deemed to be a matter for EWG and was thus split off into issue 2500.

Proposed resolution (September, 2021):

Change 7.6.1.3 [expr.call] paragraph 6 as follows:

Calling a function through an expression whose function type **E** is different from the function type **F** of the called function's definition results in undefined behavior **unless the type “pointer to F” can be converted to the type “pointer to E” via a function pointer conversion (7.3.14 [conv.fctptr])**. **[Note: The exception applies when the expression has the type of a potentially-throwing**

function, but the called function has a non-throwing exception specification, and the function types are otherwise the same. —end note]

2490. Restrictions on destruction in constant expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jiang An **Date:** 2021-05-04

According to 7.7 [expr.const] paragraph 6,

For the purposes of determining whether an expression E is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in 20.10.10.2 [allocator.members], where T is a literal type, does not disqualify E from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to `std::destroy_at`, `std::ranges::destroy_at`, `std::construct_at`, or `std::ranges::construct_at` does not disqualify E from being a core constant expression unless:

- for a call to `std::construct_at` or `std::ranges::construct_at`, the first argument, of type T^* , does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of E , or the evaluation of the underlying constructor call disqualifies E from being a core constant expression, or
- for a call to `std::destroy_at` or `std::ranges::destroy_at`, the first argument, of type T^* , does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of E , or the evaluation of the underlying destructor call disqualifies E from being a core constant expression.

There are, however, no specific restrictions in 7.7 [expr.const] regarding destructor or pseudo-destructor calls. In particular, a `constexpr` destructor can be called for any object, regardless of how it was constructed or the start of its lifetime, and similarly for pseudo-destructor calls. This seems inconsistent.

If those restrictions are added, would the specific restrictions on library destruction facilities still be needed?

Notes from the August, 2021 teleconference:

CWG agreed that since trivial destructors and pseudo-destructors are now considered to end the lifetime of the object for which they are called, they should be prohibited from being invoked for a runtime object in a constant expression.

Proposed resolution (August, 2021):

1. Change 7.7 [expr.const] paragraph 5 as follows:

An expression E is a *core constant expression* unless the evaluation of E , following the rules of the abstract machine (6.9.1 [intro.execution]), would evaluate one of the following:

- ...
- a modification of an object (7.6.19 [expr.ass], 7.6.1.6 [expr.post.incr], 7.6.2.3 [expr.pre.incr]) unless it is applied to a non-volatile lvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E ;

- **an invocation of a destructor (11.4.7 [class.dtor]) or a function call whose postfix-expression names a pseudo-destructor (7.6.1.3 [expr.call]), in either case for an object whose lifetime did not begin within the evaluation of E ;**
- a *new-expression* (7.6.2.8 [expr.new]), unless...

2. Change 7.7 [expr.const] paragraph 6 as follows, merging the single remaining bulleted item into the running text of the paragraph:

For the purposes of determining whether an expression E is a core constant expression, the evaluation of a call to a member function of `std::allocator<T>` as defined in 20.10.10.2 [allocator.members], where T is a literal type, does not disqualify E from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression. Similarly, the evaluation of a call to `std::destroy_at`, `std::ranges::destroy_at`, `std::construct_at`, or `std::ranges::construct_at` does not disqualify E from being a core constant expression unless:

- ~~for a call to `std::construct_at` or `std::ranges::construct_at`, the first argument, of type T^* , does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of E , or the evaluation of the underlying constructor call disqualifies E from being a core constant expression, or~~
- ~~for a call to `std::destroy_at` or `std::ranges::destroy_at`, the first argument, of type T^* , does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of E , or the evaluation of the underlying destructor call disqualifies E from being a core constant expression.~~

2491. Export of typedef after its first declaration

Section: 10.2 [module.interface] **Status:** ready **Submitter:** Richard Smith **Date:** 2021-04-16

According to 10.2 [module.interface] paragraph 6,

A redeclaration of an entity or *typedef-name* x is implicitly exported if x was introduced by an exported declaration; otherwise it shall not be exported. [Example 4:

```
export module M;
struct S { int n; };
typedef S S;
export typedef S S; // OK, does not redeclare an entity
export struct S;    // error: exported declaration follows non-exported declaration
```

—end example]

The normative text says that exporting a typedef that was not exported on its first declaration is ill-formed, but the example does so and states that it is “OK”. This is a contradiction that was introduced by the changes in paper P1787R6; the previous normative text supported the usage in the example.

(See also [editorial issue 4540](#).)

Proposed resolution, August, 2021:

Change 10.2 [module.interface] paragraph 6 as follows:

A redeclaration of an entity ~~or typedef-name~~ x is implicitly exported if x was introduced by an exported declaration; otherwise it shall not be exported.

2496. *ref-qualifiers* and virtual overriding

Section: 11.7.3 [class.virtual] **Status:** ready **Submitter:** Jens Maurer **Date:** 2021-06-16

According to 11.7.3 [class.virtual] paragraph 2,

If a virtual member function F is declared in a class B , and, in a class D derived (directly or indirectly) from B , a declaration of a member function G corresponds (6.4.1 [basic.scope.scope]) to a declaration of F , ignoring trailing *requires-clauses*, then G overrides¹⁰⁵ F .

This is different from C++20, where G was considered to hide, rather than to override, F if the *ref-qualifiers* of the declarations are different. This unintentional change could be addressed in one of two ways. To restore the C++20 behavior, the cited paragraph could be amended to:

...a declaration of a member function G corresponds (6.4.1 [basic.scope.scope]) to a declaration of F , ignoring trailing *requires-clauses*, **and has the same *ref-qualifier* (if any)**, then G overrides¹⁰⁵ F .

Alternatively, such a situation could be regarded as an ill-formed attempt to override the base class function, which could be specified by adding the following as a new paragraph preceding 11.7.3 [class.virtual] paragraph 7:

The *ref-qualifier*, or lack thereof, of an overriding function shall be the same as that of the overridden function.

The return type of an overriding function shall be either identical to the return type of the overridden function or *covariant*...

Notes from the August, 2021 teleconference:

CWG preferred the second option.

Proposed resolution, August, 2021:

Add the following as a new paragraph preceding 11.7.3 [class.virtual] paragraph 7:

The *ref-qualifier*, or lack thereof, of an overriding function shall be the same as that of the overridden function.

The return type of an overriding function shall be either identical to the return type of the overridden function or *covariant*...